

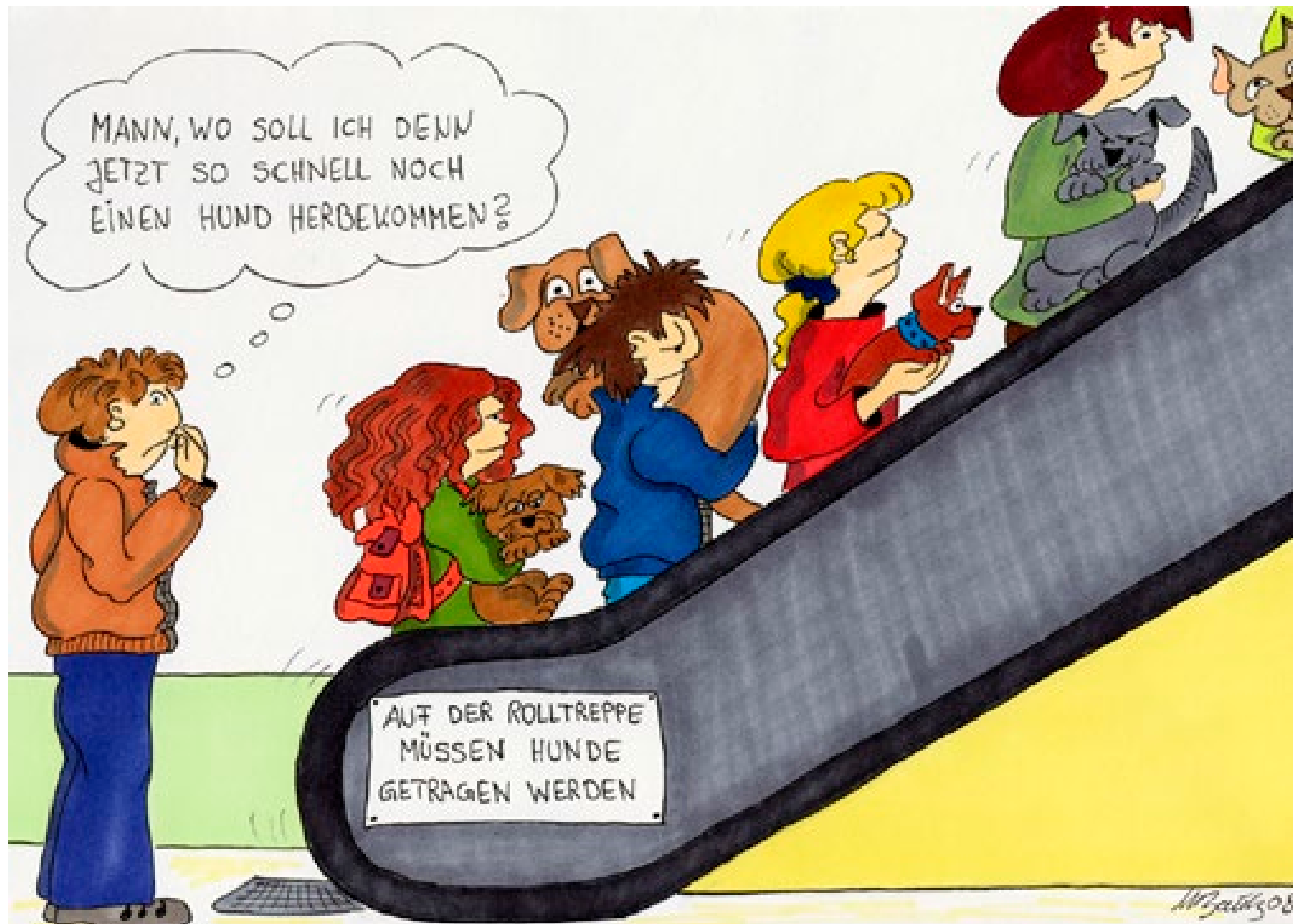
Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

FORMALE SEMANTIK



Motivation

- Für jedes Programm muss festgelegt werden, was passiert, wenn es ausgeführt wird.
 - Das ist die “Bedeutung” eines Programms
- Definition einer Programmiersprache:
 - Token, reguläre Ausdrücke $\Rightarrow$ Lexikalische Analyse
 - Grammatik $\Rightarrow$ Syntaktische Analyse
 - Semantik $\Rightarrow$ Kodeerzeugung und Optimierung

The Java Language Specification

<https://docs.oracle.com/javase/specs/jls/se21/html/>

15.1 Evaluation, Denotation, and Result

When an expression in a program is evaluated (executed), the result denotes one of three things:

- A variable ([§4.12](#)) (in C, this would be called an lvalue)
- A value ([§4.2](#), [§4.3](#))
- Nothing (the expression is said to be void)

Evaluation of an expression can also produce side effects, because expressions may contain embedded assignments, increment operators, decrement operators, and method invocations. An expression denotes nothing if and only if it is a method invocation ([§15.12](#)) that invokes a method that does not return a value, that is, a method declared void ([§8.4](#)). Such an expression can be used only as an expression statement ([§14.8](#)), because every other context in which an expression can appear requires the expression to denote something. An expression statement that is a method invocation may also invoke a method that produces a result; in this case the value returned by the method is quietly discarded.

Semantik von Programmiersprachen

- Es gibt verschiedene Spezifikationsmethoden, die für unterschiedliche Aufgabe besser geeignet sind, z.B. für
 - Beweise
 - Komplexitätsabschätzungen
 - Grundlage für die Implementierung



Andere Spezifikationsmethoden

- **Operationelle Semantik**
 - beschreibt die Programmbedeutung mittels Ausführungsregeln (einer mathematischen Maschine)
 - nützlich für die Implementierung von Programmiersprachen
 - siehe z.B. die Spezifikationen von MAMA und TRAM
- **Denotationelle Semantik**
 - Die Bedeutung eines Programms ist eine mathematische Funktion
 - elegant, aber meist sehr kompliziert
- **Axiomatische Semantik**
 - Das Programmverhalten wird mit Hilfe von logischen Formeln beschrieben:
 - “Falls die Ausführung in einem Zustand beginnt, der X erfüllt, dann endet sie in einem Zustand der Y erfüllt.”
wobei X und Y logische Formeln sind.
 - Grundlage vieler Programmverifikationssysteme
- **Action Semantics**
 - Bietet vordefinierte Operatoren, die fundamentalen Programmierkonzepten entsprechen.
 - Lesbare Semantik
- **Translational Semantics (Übersetzungssemantik?)**
 - Semantik wird definiert durch Übersetzungsregeln in eine andere Programmiersprache, deren Semantik definiert ist.
 - Geeignet für Bootstrapping

Syntax einer einfachen, imperative Sprache: SIMP

PROGRAM	::=	$STATEMENT \mid STATEMENT \boxed{;} PROGRAM$
STATEMENT	::=	$ASSIGNMENT \mid LOOP$
ASSIGNMENT	::=	$VAR \boxed{:=} EXPRESSION$
EXPRESSION	::=	$VAR \mid INT \mid EXPRESSION \boxed{-} EXPRESSION$
LOOP	::=	$\boxed{\text{while}} BOOLEXP \boxed{\text{do}} PROGRAM \boxed{\text{od}}$
BOOLEXP	::=	$EXPRESSION \boxed{>} EXPRESSION$

$X := 10;$

$\text{while } X > 0 \text{ do } X := X - 1 \text{ od}$

Denotational Semantics of SIMP

Denotational Semantics maps each program construct to a mathematical object, its denotation. The mapping is defined by semantic equations. In our example the denotations are functions from bindings to bindings. Traditionally, denotation brackets $\llbracket \cdot \rrbracket$ are used to represent these functions, or in other words, if p is a program, then $\llbracket p \rrbracket$ is the denotation of p .

$$\llbracket c_0; c_1 \rrbracket = \llbracket c_1 \rrbracket \circ \llbracket c_0 \rrbracket$$

$$\llbracket x := e \rrbracket = \lambda b. b[x/v] \quad \text{where } v = \llbracket e \rrbracket b$$

$$\llbracket \text{while } e \text{ do } c \text{ od} \rrbracket = \lambda b. \begin{cases} \llbracket c; \text{while } e \text{ do } c \text{ od} \rrbracket b & \text{if } \llbracket e \rrbracket b = \text{true} \\ b & \text{otherwise} \end{cases}$$

Unfolding $\llbracket c; \text{while } e \text{ do } c \text{ od} \rrbracket$ yields $\llbracket \text{while } e \text{ do } c \text{ od} \rrbracket \circ \llbracket c \rrbracket$

$$\Gamma(\psi) = \lambda b. \text{if } \llbracket e \rrbracket b = \text{true} \text{ then } (\psi \circ \llbracket c \rrbracket)b \text{ else } b$$

and thus

$$\llbracket \text{while } e \text{ do } c \text{ od} \rrbracket = \text{fix}(\Gamma)$$

Axiomatic Semantics of SIMP

Axiomatic semantics use partial correctness assertions (aka. Hoare Triple) of the form $\{A\} c \{B\}$. The interpretation of such an assertion is: “If property A holds before the execution of program c and the execution of c terminates, then property B holds after its execution.” The semantics of SIMP is described by the following inference rules:

$$\textcircled{1} \quad \frac{\{A\} c_0 \{B\} \quad \{B\} c_1 \{C\}}{\{A\} c_0; c_1 \{C\}}$$

$$\textcircled{2} \quad \frac{}{\{B[x/e]\} x := e \{B\}}$$

$$\textcircled{3} \quad \frac{\{A \wedge b\} c \{A\}}{\{A\} \text{ while } b \text{ do } c \text{ od } \{A \wedge \neg b\}}$$

$$\textcircled{4} \quad \frac{(A \Rightarrow A') \quad \{A'\} c \{B'\} \quad (B' \Rightarrow B)}{\{A\} c \{B\}}$$

Often, axiomatic semantics is designed to prove properties of programs, but does not necessarily provide enough information to implement a language based on the specification. In this case, the actual semantics of the programming language is only one model for the given axiomatic semantics.

$$\textcircled{1} \frac{\{A\} \ c_0 \ \{B\} \quad \{B\} \ c_1 \ \{C\}}{\{A\} \ c_0; c_1 \ \{C\}}$$

$$\textcircled{4} \frac{(A \Rightarrow A') \quad \{A'\} \ c \ \{B'\} \quad (B' \Rightarrow B)}{\{A\} \ c \ \{B\}}$$

$$\textcircled{2} \frac{}{\overline{\{B[x/e]\} \ x:=e \ \{B\}}}$$

$$\textcircled{3} \frac{\{A \wedge b\} \ c \ \{A\}}{\{A\} \ \text{while } b \text{ do } c \text{ od } \{A \wedge \neg b\}}$$

$\{x > 0\} \ \mathbf{y=99; \ x=x-7} \ \{x > -10\}$

Translation Semantics of SIMP

The semantics of a programming language can be defined by translating its constructs to constructs in another programming language with a given semantics.

$$\text{trans}(x := e) = \text{trans}(e).\langle \text{STORE } x \rangle$$

$$\text{trans}(\text{while } e \text{ do } c \text{ od}) =$$

let

$$l1 = \text{newlabel}(), l2 = \text{newlabel}(),$$

$$e' = \text{trans}(e), c' = \text{trans}(c)$$

in

$$\langle l1 \rangle.e'.\langle \text{IFFALSE } l2 \rangle.c'.\langle \text{GOTO } l1, l2, \text{NOP} \rangle$$

Operational Semantics of SIMP

We define a mathematical machine to give the semantics of SIMP. Basically the SIMP machine consists of configurations and the transition function.

$$M_{SIMP} = (K, K^f, PROGRAM, \delta)$$

A configuration is a pair of a program rest and a binding of variables.

$$K = PR \times B$$

$$\delta(p, b) = (p', b[x/I(b, e)])$$

if p is of the form $x := e; p'$ where x is a variable name, e an expression and p' a program rest and $I(b, e)$ is defined.

$$\delta(p, b) = \begin{cases} (p_1; p, b) & \text{if } I(b, e) = \text{true} \\ (p', b) & \text{if } I(b, e) = \text{false} \end{cases}$$

if p is of the form **while** e **do** p_1 **od**; p' where e is a boolean expression, p_1 a program and p' a program rest.

Here $I(b, e)$ is the interpretation of the expression e in the binding b .

Operational Semantics of SIMP

A computation step is defined as follows:

Let $k, k' \in K$

$$k \succ k' \Leftrightarrow \delta(k) = k'$$

A computation is a possibly infinite sequence $\gamma = (k_1, k_2, \dots)$ of configurations where:

$$k_i \succ k_{i+1} \text{ for all } i$$

Finally, a terminating computation is a sequence $\gamma = (k_1, k_2, \dots, k_n)$ of configurations where:

$$k_1 \succ k_2 \succ \dots \succ k_n, \forall i < n : k_i \notin K^f \text{ and } k_n \in K^f$$

The semantics of a program p is the set of all computations $(k_1, k_2, \dots)$ where $k_1 = (p, \emptyset)$ and $\emptyset$ is the empty binding.

Structural Operation Semantics of SIMP

If we represent configurations as terms, then the semantics of SIMP can be given by inference rules.

$$\frac{(b,e) \rightarrow v}{(x:=e;p', b) \rightarrow (p', b[x/v])}$$

$$\frac{(c, b) \rightarrow (c', b') \quad (c';p, b') \rightarrow (\epsilon, b'')}{(c;p, b) \rightarrow (\epsilon, b'')}$$

$$\frac{(b,e) \rightarrow true}{(\text{while } e \text{ do } p \text{ od};p', b) \rightarrow (p;\text{while } e \text{ do } p \text{ od};p', b)}$$

$$\frac{(b,e) \rightarrow false}{(\text{while } e \text{ do } p \text{ od};p', b) \rightarrow (p', b)}$$

Small Step Semantics:
Configuration $\rightarrow$ Configuration

The semantics of a program p is the set of all pairs (b, b') where $(p, b) \rightarrow (\epsilon, b')$ is provable. Note, that if a program p does not terminate for a given binding b , one cannot prove $(p, b) \rightarrow (\epsilon, b')$ for any b' .

$$\frac{(b,e) \rightarrow true}{(\text{while } e \text{ do } p \text{ od}, b) \rightarrow (p; \text{while } e \text{ do } p \text{ od}, b)}$$

$$\frac{(b,e) \rightarrow false}{(\text{while } e \text{ do } p \text{ od}, b) \rightarrow (\epsilon, b)}$$

$$\frac{(c, b) \rightarrow (c', b') \quad (c'; p, b') \rightarrow (\epsilon, b'')}{(c; p, b) \rightarrow (\epsilon, b'')} \quad \textcircled{1}$$

$$\frac{(b,e) \rightarrow v}{(x := e, b) \rightarrow (\epsilon, b[x/v])} \quad \textcircled{2}$$

Beweisbaum:

$$\begin{array}{c} \textcircled{2} \frac{(99, \{\}) \rightarrow 99}{(y=99, \{\}) \rightarrow (\epsilon, \{y \vdash 99\})} \quad \frac{(y-7, \{y \vdash 99\}) \rightarrow 92}{(x=y-7, \{y \vdash 99\}) \rightarrow (\epsilon, \{y \vdash 99, x \vdash 92\})} \quad \textcircled{2} \\ \hline (\mathbf{y=99; x=y-7, \{\}}) \rightarrow (\epsilon, \{y \vdash 99, x \vdash 92\}) \quad \textcircled{1} \end{array}$$

Structural Operation Semantics of SIMP

$$\frac{(b,e) \rightarrow v \quad (p, b[x/v]) \rightarrow b'}{(x:=e; p, b) \rightarrow b'}$$

$$\frac{(b,e) \rightarrow true \quad (p; \text{while } e \text{ do } p \text{ od}; p', b) \rightarrow b'}{(\text{while } e \text{ do } p \text{ od}; p', b) \rightarrow b'}$$

$$\frac{(b,e) \rightarrow false \quad (p', b) \rightarrow b'}{(\text{while } e \text{ do } p \text{ od}; p', b) \rightarrow b'}$$

Big Step Semantics:
Configuration $\rightarrow$ (Final) Binding

$$\frac{(b,e) \rightarrow v \quad (p, b[x/v]) \rightarrow b'}{(x:=e; p, b) \rightarrow b'} \quad \textcircled{1}$$

$$\frac{(b,e) \rightarrow \text{true} \quad (p; \text{while } e \text{ do } p \text{ od}; p', b) \rightarrow b'}{(\text{while } e \text{ do } p \text{ od}; p', b) \rightarrow b'} \quad \textcircled{2}$$

$$\frac{(b,e) \rightarrow \text{false} \quad (p', b) \rightarrow b'}{(\text{while } e \text{ do } p \text{ od}; p', b) \rightarrow b'} \quad \textcircled{3}$$

$$\frac{(99, \{\}) \rightarrow 99 \quad \frac{(y-7, \{y \vdash 99\}) \rightarrow 92 \quad (\epsilon, \{y \vdash 99, x \vdash 92\}) \rightarrow \{y \vdash 99, x \vdash 92\}}{(x=y-7, \{y \vdash 99\}) \rightarrow \{y \vdash 99, x \vdash 92\}}}{(y=99; x=y-7, \{\}) \rightarrow \{y \vdash 99, x \vdash 92\}} \quad \textcircled{1}$$

Action Semantics of SIMP

The notation of action semantics provides operators which correspond to fundamental concepts of programming languages, these operators are called actions and action combinators, and their semantics is given by SOS rules. Similar to denotational semantics, the action semantics of a programming language is given by semantic equations:

- (1) $\text{execute}[\![X := E]\!] = \begin{array}{|l} \text{evaluate } E \\ \text{then store the value in the cell bound to } X \end{array}$
- (2) $\text{execute}[\![\text{"while" } E \text{ "do" } C \text{ "od" }]\!] = \text{unfolding} \begin{array}{|l} \text{evaluate } E \\ \text{then} \\ \text{execute } C \text{ then unfold} \\ \text{else} \\ \text{complete} \end{array}$

SOS of Action Notation (Small Step)

The formal definitions of some of the actions and action combinators in the above action semantic description are given below. Here t, t', t_i are bindings for transient data, b, b', b_i are bindings of scoped data and s, s', s_i are stores. Execution of an action can complete or fail, this is indicated by the outcome status o, o' .

$$\frac{(t, b, s \rightarrow a[\text{unfold/unfolding } a]) \Rightarrow (o', t', b', s')}{(t, b, s \rightarrow \text{unfolding } a) \Rightarrow (o', t', b', s')}$$

unfolding a proceeds by replacing every occurrence of `unfold` in the action a by `unfolding  $a$` and then executing the resulting action notation term.

$$\frac{(\{\}, b, s \rightarrow a_1) \Rightarrow (o_1, t_1, b_1, s_1)}{(\{0 \Rightarrow \text{true}\}.t, b, s \rightarrow a_1 \text{ else } a_2) \Rightarrow (o_1, t_1, b_1, s_1)}$$

$$\frac{(\{\}, b, s \rightarrow a_2) \Rightarrow (o_2, t_2, b_2, s_2)}{(\{0 \Rightarrow \text{false}\}.t, b, s \rightarrow a_1 \text{ else } a_2) \Rightarrow (o_2, t_2, b_2, s_2)}$$

Schlussbemerkungen

- Die meisten Programmiersprachen haben keine präzise definierte operationelle Semantik, sondern bestensfalls eine **Referenzimplementierung**
- Für Portierbarkeit von Programmen ist eine genaue Semantik wesentlich.
- Die Semantik von Java wurde **in natürlicher Sprache** definiert, siehe “The Java Language Specification, Java SE 21 Edition”
(<https://docs.oracle.com/javase/specs>, 855 Seiten)
- Viele Wissenschaftler haben später formale Semantiken für **Teilsprachen** von Java entwickelt.

